guildtrek

PYTHON FOR AI AND DATA ENGINEERING · DEPENDABLE

Dependable

Reliable and Trustworthy. Build it so it holds up, prove it holds up, and keep proving it while it runs.

Reliability is the promise, stated in numbers. Trustworthiness is how you set the number, defend it by demonstration, and operate it with monitoring. Neither works without the other.

What we will learn today

THE SEVEN QUALITIES Performant · Scalable · Maintainable · **Reliable** · Interoperable · **Trustworthy** · Secure

TODAY two qualities that are one job: dependability.

Define and measure THE PROMISE, IN NUMBERS	What reliability and trustworthiness are, why they interlock, and the metrics with their acceptable ranges.
Level 1, engineering BUILD TO THE NUMBER	What each metric needs from the code: timeouts, idempotency, bulkheads, bounded memory, with demos.
Level 2, architecture MAKE THE NUMBER POSSIBLE	What each metric needs from the system's shape: redundancy, topology, scaling, deployment, DR.
Level 3, the live lab DEFINE, DEFEND, OPERATE	reviews-insight run through the loop, scenario by scenario, with continuous monitoring.

The loop: define, defend, observe

Every reliability metric runs through the same three steps. This loop is trustworthiness applied to reliability, and it is how the whole topic is organized.

1. Define

Agree the target up front and write it down as an **SLO**: 99.9 percent available, p99 under 300 ms, RPO zero. A target nobody set is one nobody can be held to.

2. Defend

Demonstrate you meet it **before it ships**: a test, a benchmark, a fault-injection run, a soak. Proof, not a promise.

3. Observe

Keep proving it **while it runs** (operate with continuous monitoring): logs, metrics, and traces on a dashboard against the SLO, and an alert when the target is at risk.

WHO DOES WHAT Define is architecture-led, the target and shape. Defend and Observe are where engineering proves the number and watches it. Miss any step and the number is a claim, not a guarantee.

Two qualities, one job

Reliable, the promise

The running app keeps giving correct results, stays available, and loses no data, when a dependency is down, after weeks, and under real load. It is **measured, not asserted**.

Trustworthy, the proof

The discipline that turns a reliability number into a guarantee: **define** the target, **defend** it by demonstration, **operate** it with monitoring.

Define the target (SLO) → Defend by demonstration → Operate with monitoring

WHY TOGETHER Reliability without proof is a hope; proof over fragile code proves the wrong thing. Every metric lives inside this loop, and the loop is trustworthiness applied to reliability.

Co-owned by architect and engineer

Solution architect

Sets the **target and shape**: the SLO, redundancy, topology, delivery guarantee, deploy and recovery strategy. Owns *what dependable means here*.

Engineer

Builds **to the target** and emits the signals: timeouts, idempotency, health endpoints, metrics and logs. Owns *the code that hits or misses the number*.

THE SEAM WHERE IT BREAKS The **band-aid**: "the platform will retry it" over a non-idempotent call, so it runs twice. The **premature build**: a retry queue before the delivery guarantee is fixed. The metric is the shared contract; fill both columns before the code starts.

THE METRICS

What the numbers actually mean

You cannot design for a target you have not stated, or state one without knowing what it buys. Availability, latency, success rate, throughput, MTBF and MTTR, durability, and the reliability of change.

Availability: the nines

Target	Downtime / year	/ month	/ day	Who aims here
99% (two nines)	3.65 days	7.3 h	14.4 min	internal tools, batch
99.9% (three nines)	8.8 h	43.8 min	1.44 min	a typical business service
99.99% (four nines)	52.6 min	4.4 min	8.6 s	important customer-facing
99.999% (five nines)	5.26 min	26 s	0.86 s	payments, telecom

WHAT IT MEANS Five nines is about five minutes of downtime a whole year: full redundancy, automatic failover, no manual step. Pick the lowest number the business truly needs, because each nine multiplies the cost.

Latency: percentiles, not averages

Read the tail

Never state latency as an average; it hides the slow requests. Use **p50, p95, p99, p99.9**.

p99 = 200 ms

99 of 100 requests finish under 200 ms; the slowest 1 percent take longer. At a million a day that tail is **10,000 real requests**.

`HEALTHY RANGES (INTERACTIVE API)` p50 under 100 ms, p95 under 300 ms, p99 under about 1 s. The tail is where reliability trouble shows first, so it is the number to watch.

Success rate and the error budget

The SLI

The fraction of requests that succeed, stated as an SLO: **"99.9 percent succeed over 30 days."** That is 1 failure in 1,000.

The error budget

The part you may fail, about **1,000 requests** a month at a million served. Spend it on shipping features; when it runs low, stop and fix reliability.

WHAT IT MEANS The budget turns "how reliable is reliable enough" from an argument into a number both roles agreed to, and it decides when the team ships versus hardens.

Throughput, and MTBF versus MTTR

Throughput and headroom

Sustained load while holding the SLOs. Run at **60 to 70 percent** of measured capacity, so a burst or a lost instance still fits. At 100 percent, the next spike is an outage.

MTBF and MTTR

Availability = MTBF / (MTBF + MTTR). Recovery time matters as much as failure rate: cutting MTTR from 1 hour to 1 minute lifts availability more cheaply than doubling MTBF.

ACCEPTABLE RANGES Keep real headroom, not a full pipe. Target automated recovery in minutes; the elite benchmark for restoring service is under one hour. Design for fast recovery first, rare failure second.

Durability, RPO and RTO, and the reliability of change

Durability

The chance accepted data is never lost, in nines. Object stores quote **eleven**: about one object lost in a hundred billion a year.

RPO and RTO

RPO = data you can afford to lose (zero for money, minutes for telemetry). **RTO** = how long a restore may take. RPO drives backups; RTO drives the DR design.

Reliability of change

Change failure rate under 15 percent, plus frequent, reversible deploys. Safe change is itself a reliability property.

WHAT IT MEANS A service that ships small changes often, fails under 15 percent of the time, and recovers in under an hour is more dependable than one that rarely changes and breaks hard when it does.

LEVEL 1 · THE ENGINEERING STANDPOINT

What each metric needs from the code

Build to the number, and demonstrate it. The resilience patterns are the vocabulary: timeout, fallback, retry with idempotency, circuit breaker, bulkhead, load shedding, bounded memory, transactions.

How Python supports reliability

You do not write reliability from scratch each time. The language carries it, and a mature ecosystem packages the patterns. We build the core by hand here, then reach for these in production.

Language and types

- `try / except / else / finally`, `contextlib`, `with`
- `asyncio.gather(..., return_exceptions=True)`
- `typing` + `@dataclass(frozen=True)`
- **mypy strict**: a class of bug caught before the code runs

Prove and protect

- **pytest**, **hypothesis** (auto edge cases)
- `icontract`, `deal` (pre/postconditions)
- `tenacity`, `stamina` (retry + backoff)
- `pybreaker`, `aiolimiter` (breaker, throttle)

Validate and see

- `pydantic`, `pandera`, `great_expectations`
- `opentelemetry` (traces, metrics, logs)
- `prometheus_client`, `structlog`

AI OUTPUT, FOR THE WORK THIS COURSE FEEDS guardrails-ai and instructor enforce a JSON schema on a model's response; deepeval, ragas, and trulens measure hallucination and answer quality. Reliability applied to a probabilistic dependency.

Availability: timeout, fallback, readiness

AVAILABILITY a slow dependency, not a crash, is the usual cause.

```python
async def enrich_or_degrade(review, client):
    try:
        r = await client.post("http://enrichment/insight", json=review, timeout=1.0)
        return r.json()
    except (httpx.TimeoutException, httpx.HTTPError):
        return {**review, "insight": None, "enriched": False}   # degrade, do not fail
```

DEMO · DEGRADE_ON_TIMEOUT A mock enrichment times out; the service returns the review un-enriched instead of hanging or erroring. The core path stays available against a dead dependency.

Success rate: retry, but only if safe

SUCCESS RATE retry transient failures; make the retry safe.

```
def submit_review(review, key, store):
    if key in store:          # idempotency key: twice has the same effect as once
        return store[key]
    store[key] = {"status": "accepted", "review": review}
    return store[key]
```

```
demo_retry_idempotency:  no key  -> applied 2 times
                         with key -> applied exactly once, despite the retries
```

COMMON TRAP Retry without idempotency. A retried call whose response was lost applies the effect twice: a double-counted review, or a double charge on the money track.

Throughput, MTBF, durability in code

Throughput

Stream in constant memory, give each dependency a **bulkhead** pool, and **shed load** early (a fast 429) under overload.

MTBF

No unbounded caches, no leaks. The reliability-in-the-small habits, carried forward, are what let it run for months.

Durability

Write before you acknowledge, wrap multi-step writes in a **transaction**, and dedupe with an idempotency key.

DEMOS demo_load_shedding (accepted work keeps its SLO), demo_soak_flat (bounded memory stays flat while an unbounded cache climbs), demo_crash_in_the_middle (the store holds the whole record or none, never a half).

Observe: the signals the code emits

The engineering half of the Operate step. A metric you cannot see cannot be defended in production, so the code emits the signals that let the system be watched.

Structured logs

One JSON line per event, leveled, carrying the `trace_id`, so a request's whole path is one query, not a guess.

SLI metrics

Request **rate**, **errors**, and **duration** (RED), exposed at `/metrics`. These are the SLOs, measured.

Health signals

`/health` for liveness and `/ready` for real capacity, so the platform routes only to instances that can actually serve.

IN THE LIVE LAB The `observe` middleware times each request, records the SLIs, and logs a structured line with the trace id; `demo_structured_logging` follows one request end to end.

LEVEL 2 · THE ARCHITECTURE STANDPOINT

What each metric needs from the system's shape

The same metrics from the other side of the contract: what the architecture must decide and arrange so the code can reach the target. Each is addressed at many points.

Availability: redundancy at every layer

- **Redundant instances** behind a balancer
- **N+1 spare capacity**: hold the SLO with one gone
- **Multiple zones**: survive a zone failure
- **Multiple regions**: survive a region loss

- **Balancer reads readiness**: pull hung instances
- **Stateless tier**: any instance serves any request
- **Redundant dependencies**: replica with failover
- **Degradation decision**: partly up beats all down

SCENARIO A zone goes dark; two of three zones keep serving and the SLO holds. Engineering built the readiness endpoint; architecture placed the instances so there is somewhere healthy to route to.

Latency and success rate, by design

Latency: remove waiting

- Cache tier in front of the store
- Read replicas; regional and edge deploys
- Precompute; move off-path work to a queue

Success: cut off failure

- Bulkheads isolate a slow dependency
- Decouple a fragile one via a queue or stream
- Choose the delivery guarantee per data type

SCENARIO The enrichment service is chronically flaky. Architecture moves it off the request path onto an event stream: the review is served immediately and enriched asynchronously, under an at-least-once-plus-dedupe guarantee.

Throughput, recovery, durability, by design

Throughput

- Horizontal autoscaling on an SLI
- Partition and shard the data
- Queue to level spikes; shed at the gateway

MTBF and MTTR

- Canary or blue-green rollout
- Automatic rollback gate; self-healing
- Expand-then-contract migrations; chaos drills

Durability

- Sync or async replication
- Backups sized to RPO
- DR sized to RTO: warm-standby, active-active

SCENARIO A region is lost; a warm standby in a second region takes over within the one-hour RTO, and synchronous replication on the money path means RPO zero.

Observe: monitor, alert, and close the loop

The architecture half of the Operate step. Signals are only useful if someone watches them against a target and is told the moment it slips.

Dashboards vs SLOs

Each SLI charted against its SLO, so "how are we doing" is a glance, and the error budget is visible as it burns.

Burn-rate alerts

Page when the budget is burning fast enough to threaten the objective, not on every blip. Alerting on noise trains people to ignore it.

Traces and synthetics

Distributed tracing (OpenTelemetry) across services, plus a **synthetic check** calling prod on a schedule to catch failure first.

THE LOOP CLOSES Alert to runbook to fast recovery, and every incident becomes a regression test, so a failure seen once in production is watched for forever.

The handshake, one row per concern

Concern	Architecture decides	Engineering builds	The product defends
Availability	SLO, redundancy, zones	readiness, degradation	uptime vs SLO, synthetic check
Latency	topology, caches, regions	timeouts, caching, batching	p99 dashboard vs SLO
Success rate	isolation, decoupling, guarantee	validation, retry, idempotency	error-rate SLO, fault-injection test
Throughput	scaling, partitioning	streaming, bulkheads, shedding	load test at target volume
MTTR	rollout and rollback strategy	fast idempotent restart	timed recovery drill
Durability	replication, backups, DR	idempotency, transactions	crash and DR-restore tests

READ IT Empty middle column = a band-aid waiting to fail. Engineering filling the left alone = building ahead of a decision. Fill both, agreed up front.

A/B testing: validate a change on real traffic

Before the live lab, one more way to defend a change: test it on real users. Send some traffic to the current version (A, the control) and some to the new version (B), measure both, and keep the winner. It is the define, defend, observe loop applied to a single change.

How it works

Split live traffic A and B, measure each against the same signals, and decide on **evidence**, not opinion. Expand B only when the numbers say it is safe.

Guardrail metrics

Watch **error rate, p99 latency, and success rate** on the B slice, not just the business metric. A guardrail breach stops the experiment automatically.

Small blast radius

Only a slice sees B, so a bad change is caught on a few users before it can reach everyone. Reversible by design.

THE FAMILY A/B testing, canary, and feature flags are one idea at different grains: expose a change to a little of production, measure it against reliability guardrails, and roll forward or back on the result.

A/B testing: reliability and trust

It raises reliability

- Smaller blast radius, so a bad change hits few, not all
- Guardrails auto-rollback on an error or latency regression
- Lower **change failure rate**, the delivery metric
- The SLO is protected *during* the rollout, not just after

It raises trust

- Decisions from measured evidence, not a hunch
- **Statistical significance**: enough samples to tell signal from noise
- Reliability guardrails stay sacred while chasing a business metric
- An auditable trail: what changed, what it did, why it shipped

WHY IT BELONGS HERE A change is the most common cause of an outage. Proving it on real traffic, behind guardrails, turns "we think this is better and safe" into "we measured that it is, and it can be pulled in seconds."

LEVEL 3 · THE LIVE LAB

reviews-insight, run through the loop

Every scenario, the same three steps: define the target, defend it by demonstration, operate it with continuous monitoring. This loop is what shapes the code.

Define (SLO) → Defend (demonstration) → Operate (monitoring)

Scenario A · the enrichment dependency is slow

Define

99.9 percent on the core path. If enrichment is slow or down, serve the review un-enriched. One-second timeout.

Defend

A fault-injection test slows then fails the enrichment mock; the API still returns `enriched: false`, never hangs.

Operate

Enrichment error rate and duration as metrics, on a dashboard, with a burn-rate alert that pages in about a minute.

SCENARIO B · DUPLICATE SUBMISSION Define: same idempotency key twice, one effect, RPO zero. Defend: a test submits twice with a lost-response retry, one review stored. Operate: a duplicate-key metric flags a misbehaving client early.

Scenario C and D · long run, and bad deploy

C · runs for weeks (MTBF)

Define: memory and latency flat. **Defend:** a soak run stays flat next to an unbounded climb. **Operate:** memory and duration metrics with a trend alert.

D · bad change deployed (MTTR)

Define: canary with auto-rollback on readiness, MTTR under a minute. **Defend:** a rollback-gate demo. **Operate:** post-deploy smoke plus deployment metrics.

OPERATE IS THE OLD TOPICS Observability (structured logs with the trace id, metrics as SLIs, traces), deployability (canary, rollback), and incident response (burn alert, runbooks, incident becomes a regression test) are simply the Operate step.

Demo · the Operate step, made concrete

OPERATE `demo_structured_logging.py` · `demo_rollback_gate.py`

```
{"event": "submit", "trace_id": "9f2a", "product_id": "P1", "duration_ms": 42}
{"event": "enrich", "trace_id": "9f2a", "status": "degraded"}      <- filter by trace, see the path

rollout v2 -> readiness FAIL -> auto rollback to v1   (bad deploy never stays live)
```

WHAT IT MEANS One trace id ties a request's log lines together so a production failure is a lookup, not a guess. A failed deploy rolls itself back, so a bad change cannot stay in production.

What we covered

Define and measure

- Reliability = the promise in numbers; trustworthiness = define, defend, operate
- The metrics with ranges: nines, percentiles, budgets, MTBF/MTTR, RPO/RTO
- Co-owned by architect and engineer

Build, arrange, and run it

- Engineering: timeout, idempotency, bulkhead, bounded memory, transactions
- Architecture: redundancy, topology, scaling, rollout, DR
- The live lab: every scenario defined, defended, operated

THE REVIEW LENS For each target: who set it, which dependency can fail and what happens, is every call timed out and every write retry-safe, could you find a failure from its signals, and where is the demonstration that defends it and the monitor that watches it.

guildtrek

DEPENDABLE · COMPLETE

A number, defended and watched.

Reliability sets the target; trustworthiness defines it, defends it by demonstration, and operates it with monitoring. Architecture makes the number possible, engineering hits it, and the product carries the evidence.

Next: the live lab shapes reviews-insight scenario by scenario, then the telemetry and transactions assignments.